University Institute of Engineering

Department of Computer Science & Engineering

Experiment: 10

Student Name: Jagrath **UID:24BDA70365**

Branch: CSE **Section/Group: AIT-KRG-GP2**

Semester: 4th **Date of Performance:24/04/2026**

Subject Name: DBMS

1. **Aim of the practical:** To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

2. Tool Used:

- **Database Management System:**
  - **PostgreSQL**
- **Database Administration Tool:**
  - **pgAdmin**

3. **Objective:**

To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

4. Practical / Experimental Steps

Step 1: Create the required table with a primary key column.

Step 2: Create a sequence to generate unique key values.

Step 3: Write a trigger function to assign the sequence value before insertion.

Step 4: Define a BEFORE INSERT trigger on the table.

Step 5: Link the trigger to the trigger function.

Step 6: Insert multiple records without specifying the primary key.

Step 7: Verify that unique primary keys are generated automatically.

5. I / O Analysis

```
CREATE TABLE employees (
   id INT PRIMARY KEY,
   name VARCHAR(100),
   salary INT
);
```

```
CREATE TABLE

Query returned successfully in 126 msec.
```

```
CREATE SEQUENCE emp_seq
START 1;
```

```
CREATE SEQUENCE

Query returned successfully in 28 msec.
```

```
CREATE OR REPLACE FUNCTION assign_emp_id()
RETURNS TRIGGER AS $$
BEGIN
   -- If ID is not provided, generate it automatically
   IF NEW.id IS NULL THEN
      NEW.id := nextval('emp_seq');
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION

Query returned successfully in 129 msec.
```

```
CREATE TRIGGER before_insert_emp
BEFORE INSERT ON employees
FOR EACH ROW
EXECUTE FUNCTION assign_emp_id();
```

```
CREATE TRIGGER

Query returned successfully in 48 msec.
```

```
INSERT INTO employees (name, salary) VALUES ('Rahul', 50000);
INSERT INTO employees (name, salary) VALUES ('Anita', 60000);
INSERT INTO employees (name, salary) VALUES ('Vikas', 55000);
SELECT * FROM employees;
```

	id [PK] integer	name character varying (100)	salary integer
1	1	Rahul	50000
2	2	Anita	60000
3	3	Vikas	55000

6. Learning outcomes (What I have learnt):

- Understand the purpose and working of database triggers.
- Implement automated primary key functionality using triggers
- Ensure data integrity without manual key assignment.
- Apply trigger-based automation in real-world enterprise applications such as Amazon, Flipkart, and Oracle.